

Manual Técnico – Practica 1

Angel Emanuel Rodriguez Corado

202404856

Indice

Introducción	3
Objetivos.....	3
Público Objetivo	3
Requisitos técnicos.....	3
Hardware	3
Software	3
Lógica General del Sistema	4
Funcionamiento de los Módulos Principales	4
LexicalAnalyzer.ts	4
Token.ts.....	5
Index.ejs.....	6
PokemonParser.ts	8
errores.ejs	9
index.ts	11
analyze.route.ts	11
analyze.controller.ts	12
Automate Finito Determinista AFD	13

Introducción

Este documento describe los componentes técnicos del proyecto desarrollado en TypeScript para analizar un lenguaje específico que permite seleccionar los seis mejores Pokémons desde un archivo .pkf. El sistema incluye un analizador léxico basado en un autómata finito determinista (AFD), generación de reportes de tokens, manejo de errores, y visualización de los Pokémons seleccionados.

Objetivos

- Documentar la lógica de implementación del analizador léxico.
- Explicar la estructura del código fuente.
- Servir como guía técnica para mantenimiento o evolución del sistema.

Público Objetivo

Este manual está dirigido a desarrolladores, docentes y estudiantes que deseen entender o modificar la lógica del proyecto, o que necesiten extender su funcionalidad.

Requisitos técnicos

Hardware:

- Procesador Intel i5 o superior
- Mínimo 4 GB de RAM

Software:

- Node.js 18+
- TypeScript
- Express.js
- EJS
- Navegador web moderno
- Acceso a Internet (para consumir la PokeAPI)

Lógica General del Sistema

1. El usuario carga un archivo .pklf o edita el texto directamente en el editor.
2. Al presionar “Analizar”, el backend ejecuta un analizador léxico personalizado basado en AFD.
3. Se generan:
 - a. Tokens válidos → mostrados en tabla. o Errores léxicos → renderizados en una página HTML.
 - b. Equipo Pokémon → se seleccionan los 6 con mejores IVs y se renderiza una vista con sprites obtenidos de la PokeAPI.

Funcionamiento de los Módulos Principales

LexicalAnalyzer.ts

- Implementa el Autómata Finito Determinista (AFD).
- Lee línea por línea y analiza cada carácter.
- Clasifica lexemas en:
 - Palabras reservadas
 - Tipos de Pokémon
 - Cadenas (entre comillas)
 - Números (enteros)
 - Símbolos
- Registra tokens válidos y errores léxicos con fila y columna.
- **Se trabajaron los estados de las palabras reservadas en grupo de 10 en 10 para una mejor comprensión y análisis del automata**

Link del código:

https://github.com/Deltai-cod12/LFP_Practica1_202404856/blob/main/src/Analyzer/LexicalAnalyzer.ts

Token.ts

- Define interfaces para:
- Token: tipo, valor, fila, columna • Error: carácter, descripción, ubicación • Enumera los tipos de tokens reconocidos.

```
enum Type {
    UNKNOWN,
    PAR_OPEN,      // (
    PAR_CLOSE,     // )
    SEMICOLON,     // ;
    EQUAL,         // =
    COLON,         // :
    ASSIGN,        // :=
    BRACKET_OPEN,  // [
    BRACKET_CLOSE, // ]
    BRACE_OPEN,    // {
    BRACE_CLOSE,   // }
    NUMBER,
    STRING,
    RESERVED_WORD,
}

class Token {
    private row: number;
    private column: number;
    private lexeme: string;
    private typeToken: Type;
    private typeTokenString: string;

    constructor(typeToken: Type, lexeme: string, row: number, column: number){
        this.typeToken = typeToken;
        this.typeTokenString = Type[typeToken];
        this.lexeme = lexeme;
        this.row = row;
        this.column = column;
    }

    getRow(): number {
        return this.row;
    }

    getColumn(): number {
        return this.column;
    }

    getLexeme(): string {
        return this.lexeme;
    }

    getType(): Type {
        return this.typeToken;
    }

    getTypeTokenString(): string {
        return this.typeTokenString;
    }
}

export { Token, Type }
```

Index.ejs

Pagina principal donde se muestra la tabla de tokens y la tabla de Pokemons (si no encuentra errores)

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <title>Pokémon Selector</title>
6    <link rel="stylesheet" href="/css/estilos.css">
7  </head>
8  <body>
9    <header>
10     <nav>
11       <ul>
12         <li><button onclick="location.href='/'">Home</button></li>
13         <form id="formErrores" action="/error-report" style="display:inline;">
14           <input type="hidden" name="erroresJSON" id="erroresJSON" value='<%= JSON.stringify(errores || []) %>\'>
15           <button type="submit">Error Report</button>
16         </form>
17       </ul>
18     </nav>
19     <select id="archivoMenu">
20       <option disabled selected>Archivo</option>
21       <option value="limpiar">Limpiar Editor</option>
22       <option value="cargar">Cargar Archivo</option>
23       <option value="guardar">Guardar Archivo</option>
24     </select>
25   </li>
26 </ul>
27 </nav>
28 </header>
29
30 <main>
31   <form action="/analyze" method="POST">
32     <section id="editor-container">
33       <textarea id="editor" name="txtArea" placeholder="Escribe o carga tu archivo .pklfp aqui..."><%= codigo || '' %></textarea>
34       <button type="submit">Analizar</button>
35     </section>
36   </form>
37
38   <section id="tablaTokens-container">
39     <h2>Tabla de Tokens</h2>
40     <table id="tablaTokens">
41       <thead>
42         <tr>
43           <th>#</th>
44           <th>Fila</th>
45           <th>Columna</th>
46           <th>Lexema</th>
47           <th>Token</th>
48         </tr>
49       </thead>
50       <tbody>
51         <%= if (tokens && tokens.length > 0) { %>
52           <%= tokens.forEach((token, index) => { %>
53             <tr>
54               <td><%= index + 1 %></td>
55               <td><%= token.fila %></td>
56               <td><%= token.columna %></td>
57               <td><%= token.lexema %></td>
58               <td><%= token.token %></td>

```

```

59     </tr>
60     <% }) %>
61     <% } %>
62 </tbody>
63 </table>
64 </section>
65
66 <% if (jugadores && jugadores.length > 0) { %>
67     <% jugadores.forEach((player, playerIndex) => { %>
68         <section class="jugadorPokemon-container">
69             <h2>Jugador <%= playerIndex + 1 %>: <%= player.nombre %></h2>
70             <h3>Pokémon Registrados:</h3>
71             <table class="tablaPokemons">
72                 <thead>
73                     <tr>
74                         <th>#</th>
75                         <th>Nombre</th>
76                         <th>Tipo</th>
77                         <th>Salud</th>
78                         <th>Ataque</th>
79                         <th>Defensa</th>
80                         <th>Sprite</th>
81                     </tr>
82                 </thead>
83                 <tbody>
84                     <% player.pokemons.forEach((pokemon, index) => { %>
85                         <tr>
86                             <td><%= index + 1 %></td>
87                             <td><%= pokemon.nombre %></td>
88                             <td><%= pokemon.tipo %></td>
89                             <td><%= pokemon.salud %></td>
90                             <td><%= pokemon.ataque %></td>
91                             <td><%= pokemon.defensa %></td>
92                             <td>
93                                 <% if (pokemon.sprite) { %>
94                                     " width="64" height="64">
95                                 <% } else { %>
96                                     No disponible
97                                 <% } %>
98                             </td>
99                         </tr>
100                     <% }) %>
101                 </tbody>
102             </table>
103         </section>
104     <% }) %>
105     <% } else { %>
106         <p>No hay jugadores para mostrar.</p>
107     <% } %>
108
109 </main>
110
111 <input type="file" id="fileInput" accept=".pk1fp" style="display:none">

```

```

112
113 <script src="/js/index.js"></script>
114 </body>
115 </html>
116

```

PokemonParser.ts

Guarda a los jugadores y sus pokemons luego de realizar el análisis , también calcula los 6 mejores pokemons

```

1  import { Token } from "../token";
2  import { Type } from "../token";
3
4  export interface Pokemon {
5      nombre: string;
6      tipo: string;
7      salud: number;
8      ataque: number;
9      defensa: number;
10     ives?: number;
11     sprite?: string;
12 }
13
14 export interface Jugador {
15     nombre: string;
16     pokemons: Pokemon[];
17 }
18
19 export class PokemonParser {
20     private tokens: Token[];
21     private jugadores: Jugador[] = [];
22     private jugadorActual: Jugador | null = null;
23
24     constructor(tokens: Token[]) {
25         this.tokens = tokens;
26     }
27
28     public parse(): Jugador[] {
29         let i = 0;
30         while (i < this.tokens.length) {
31             const token = this.tokens[i];
32
33             // Detectar cabecera del jugador
34             if (
35                 token.getType() === Type.RESERVED_WORD &&
36                 token.getLexeme().toLowerCase() === "jugador" &&
37                 this.tokens[i + 1].getType() === Type.COLON &&
38                 this.tokens[i + 2].getType() === Type.STRING &&
39                 this.tokens[i + 3].getType() === Type.BRACE_OPEN
40             ) {
41                 // Si ya hay un jugador actual, agregarlo a la lista antes de crear uno nuevo
42                 if (this.jugadorActual) {
43                     this.jugadores.push(this.jugadorActual);
44                 }
45
46                 const nombreToken = this.tokens[i + 2];
47                 this.jugadorActual = {
48                     nombre: this.cleanString(nombreToken.getLexeme()),
49                     pokemons: []
50                 };
51                 i += 4; // Saltar hasta el {
52                 continue;
53             }
54
55             token.getType() === Type.STRING &&
56             this.tokens[i + 1].getType() === Type.BRACKET_OPEN &&
57             this.tokens[i + 2].getType() === Type.RESERVED_WORD &&
58             this.tokens[i + 3].getType() === Type.BRACKET_CLOSE &&
59             this.tokens[i + 4].getType() === Type.ASSIGN &&
60             this.tokens[i + 5].getType() === Type.PAR_OPEN
61         ) {
62             const nombrePokemon = this.cleanString(token.getLexeme());
63             const tipo = this.tokens[i + 2].getLexeme().toLowerCase();
64
65             const stats: Record<string, number | null> = {
66                 salud: null,
67                 ataque: null,
68                 defensa: null
69             };
70
71             i += 6; // Posicionarse dentro del bloque de estadísticas
72
73             // Leer las estadísticas
74             while (i < this.tokens.length) {
75                 if (
76                     this.tokens[i].getType() === Type.BRACKET_OPEN &&
77                     this.tokens[i + 1].getType() === Type.RESERVED_WORD &&
78                     this.tokens[i + 2].getType() === Type.BRACKET_CLOSE &&
79                     this.tokens[i + 3].getType() === Type.EQUAL &&
80                     this.tokens[i + 4].getType() === Type.NUMBER &&
81                     this.tokens[i + 5].getType() === Type.SEMICOLON
82                 ) {
83                     const nombreStat = this.tokens[i + 1].getLexeme().toLowerCase();
84                     const valorStat = parseInt(this.tokens[i + 4].getLexeme());
85
86                     if (["salud", "ataque", "defensa"].includes(nombreStat)) {
87                         stats[nombreStat] = valorStat;
88                     }
89
90                     i += 6;
91                     continue;
92                 }
93
94                 if (this.tokens[i].getType() === Type.PAR_CLOSE) {
95                     i++; // Salir del bloque del Pokémon
96                     break;
97                 }
98
99                 console.warn("Estructura inesperada dentro del bloque del Pokémon:", this.tokens[i]);
100                 break;
101             }
102
103             // Verificar que los stats estén completos antes de agregar
104             if (stats.salud !== null && stats.ataque !== null && stats.defensa !== null) {
105                 this.jugadorActual.pokemons.push({
106                     nombre: nombrePokemon,
107                     tipo,
108                     salud: stats.salud,
109                     ataque: stats.ataque,
110                     defensa: stats.defensa
111                 });
112             }
113         }
114     }

```

```

114         defensa: stats.defensa
115     });
116     } else {
117         console.warn("Pokémon "${nombrePokemon}" con estadísticas incompletas. No será agregado.");
118     }
119 }
120 continue;
121 }
122
123 i++; // Incrementar manualmente
124 }
125
126 // Agregar el último jugador encontrado si existe
127 if (this.jugadorActual) {
128     this.jugadores.push(this.jugadorActual);
129     this.jugadorActual = null;
130 }
131
132 return this.jugadores;
133 }
134
135 private cleanString(cadena: string): string {
136     return cadena.replace(/"/g, "");
137 }
138
139 // Calculo de los IVs
140
141 public calcularIVs(pokemones: Pokemon[]): (Pokemon & { iv: number })[] {
142     return pokemones.map(pokemon => {
143         const iv = ((pokemon.salud + pokemon.ataque + pokemon.defensa) / 45) * 100;
144         return { ...pokemon, iv };
145     });
146 }
147
148 // Los 6 mejores pokemons por IVs
149 public seleccionarMejoresSeis(pokemones: (Pokemon & { iv: number })[]): (Pokemon & { iv: number })[] {
150     const mejoresPorTipo = new Map<string, (Pokemon & { iv: number })>();
151
152     for (const pkm of pokemones) {
153         const tipo = pkm.tipo.toLowerCase();
154         if (!mejoresPorTipo.has(tipo)) {
155             mejoresPorTipo.set(tipo, pkm);
156         } else {
157             const existente = mejoresPorTipo.get(tipo);
158             if (pkm.iv > existente.iv) {
159                 mejoresPorTipo.set(tipo, pkm);
160             }
161         }
162     }
163
164     // Ordenar por IV descendente
165     return Array.from(mejoresPorTipo.values())
166         .sort((a, b) => b.iv - a.iv)
167         .slice(0, 6);
168 }
169 }

```

errores.ejs

Se muestra en el navegador los errores que hubieron durante el análisis

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <title>Reporte de Errores Léxicos</title>
  <link rel="stylesheet" href="/css/estilos.css" />
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><button onclick="location.href='/'">Home</button></li>
        <li><button onclick="location.href='/error-report'">Error Report</button></li>
        <li>
          <select id="archivos" disabled>
            <option disabled selected>Archivo</option>
          </select>
        </li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="reporteErrores-container">
      <h2>Reporte de Errores Léxicos</h2>

      <% if (erroresLexicos && erroresLexicos.length > 0) { %>
        <table id="tablaErrores">
          <thead>
            <tr>
              <th>#</th>
              <th>Fila</th>
              <th>Columna</th>
              <th>Lexema</th>
              <th>Descripción</th>
            </tr>
          </thead>
          <tbody>
            <% erroresLexicos.forEach((error, index) => { %>
              <tr>
                <td><%= index + 1 %></td>
                <td><%= error.fila %></td>
                <td><%= error.columna %></td>
                <td><%= error.lexema %></td>
                <td><%= error.token %></td>
              </tr>
            <% }) %>
          </tbody>
        </table>
      <% } else { %>
        <p>No se encontraron errores léxicos.</p>
      <% } %>
    </section>
  </main>
</body>
</html>

```

index.js

Se encarga de la carga del archivo, al igual que hacer la limpieza y la acción del botón analizar

Este script maneja la interacción del usuario con la interfaz. Sus funciones principales son:

- **Cargar archivo .pkllfp:** Lee el contenido del archivo seleccionado y lo coloca en el área de texto.
- **Limpiar el editor:** Borra el contenido del área de texto.
- **Guardar archivo:** Descarga el contenido actual del editor como un archivo .pkllfp.
- **Análisis léxico:**
 - Envía el texto al backend (/analyze) para su análisis. ○ Muestra los tokens válidos en una tabla.
 - En caso de errores, los envía al servidor para mostrar el reporte HTML.

```

1 document.addEventListener('DOMContentLoaded', () => {
2   const fileInput = document.getElementById('fileInput');
3   const archivoMenu = document.getElementById('archivoMenu');
4   const editor = document.getElementById('editor');
5   const analizarBtn = document.getElementById('analizar');
6   const tablaTokens = document.querySelector('#tablaTokens tbody');
7
8   archivoMenu.addEventListener('change', () => {
9     const opcion = archivoMenu.value;
10
11     if (opcion === 'limpiar') {
12       editor.value = '';
13     } else if (opcion === 'cargar') {
14       fileInput.click(); // abrir selector de archivos
15     } else if (opcion === 'guardar') {
16       const blob = new Blob([editor.value], { type: 'text/plain;charset=utf-8' });
17       const a = document.createElement('a');
18       a.href = URL.createObjectURL(blob);
19       a.download = 'archivo.pkllfp';
20       a.click();
21     }
22
23     archivoMenu.selectedIndex = 0; // reiniciar menú
24   });
25
26   fileInput.addEventListener('change', (e) => {
27     const file = e.target.files[0];
28
29     if (!file) return;
30
31     // Validar que sea archivo .pkllfp
32     if (!file.name.toLowerCase().endsWith('.pkllfp')) {
33       alert('Solo se permiten archivos con extensión .pkllfp');
34       fileInput.value = ''; // limpiar selección
35       return;
36     }
37
38     const reader = new FileReader();
39     reader.onload = function (event) {
40       editor.value = event.target.result;
41     };
42     reader.readAsText(file);
43   });
44
45 });

```

index.ts

Configura y lanza el servidor express al igual que define middelwares estáticos y motor de vistas

```
1  import express from 'express';
2  import path from 'path';
3  import analyzeRouter from './routes/analyze.route';
4
5  const app = express();
6  const PORT = 3000;
7
8  // Configuración de vistas EJS
9  app.set('views', path.join(__dirname, '../views'));
10 app.set('view engine', 'ejs');
11
12 // Archivos estáticos
13 app.use(express.static(path.join(__dirname, '../public')));
14
15 // Middleware para procesar formularios
16 app.use(express.urlencoded({ extended: true }));
17 app.use(express.json());
18
19 // Rutas
20 app.use(analyzeRouter);
21
22 // Servidor
23 app.listen(PORT, () => {
24   console.log(` Servidor corriendo en http://localhost:\${PORT}`);
25 });
26 |
```

analyze.route.ts

Recibe el texto para el análisis, renderiza el index.ejs y errores.ejs

```

1  import { Router } from 'express';
2  import { home, analyze, errorReport } from '../controllers/analyze.controller';
3
4  const analyzeRouter = Router();
5
6  analyzeRouter.get('/', home);
7  analyzeRouter.post('/analyze', analyze);
8  analyzeRouter.get('/error-report', errorReport)
9  export default analyzeRouter;

```

analyze.controller.ts

Lleva el análisis y es el que llama al AFD. También gestiona los métodos de las rutas

```

1  import { Request, Response } from 'express';
2  import { LexicalAnalyzer } from '../Analyzer/LexicalAnalyzer';
3  import { Token } from '../Analyzer/Token';
4  import { PokemonParser, Jugador } from '../Analyzer/PokemonParser';
5  import { obtenerSprite } from '../utils/pokeapi';
6
7  let lastLexicalErrors: { fila: number; columna: number; lexema: string; token: string }[] = [];
8
9  export const home = (req: Request, res: Response) => { ...
17 };
18
19 export const analyze = async (req: Request, res: Response) => {
20   const input = req.body.txtArea || '';
21
22   const lexicalAnalyzer = new LexicalAnalyzer();
23   const tokenList = lexicalAnalyzer.scanner(input);
24   const rawErrorList = lexicalAnalyzer.getErrorList();
25
26   lastLexicalErrors = rawErrorList.map((errorToken: Token) => ({
27     fila: errorToken.getRow(),
28     columna: errorToken.getColumn(),
29     lexema: errorToken.getLexeme(),
30     token: errorToken.getTypeTokenString()
31   }));
32
33   const tokensToSend = tokenList.map(token => ({
34     fila: token.getRow(),
35     columna: token.getColumn(),
36     lexema: token.getLexeme(),
37     token: token.getTypeTokenString()
38   }));
39
40   let jugadores: Jugador[] = [];
41
42   function calcularIV(pokemon: { salud: number; ataque: number; defensa: number }) {
43     return ((pokemon.salud + pokemon.ataque + pokemon.defensa) / 45) * 100;
44   }
45
46   if (lastLexicalErrors.length === 0) {
47     try {
48       const parser = new PokemonParser(tokenList);
49       jugadores = parser.parse();
50
51       jugadores = await Promise.all(jugadores.map(async jugador => {
52         const pokemonesConIV = await Promise.all(jugador.pokemones.map(async p => ({
53           ...p,
54           iv: calcularIV(p),
55           sprite: await obtenerSprite(p.nombre.toLowerCase()) || ''
56         })));
57
58         pokemonesConIV.sort((a, b) => b.iv - a.iv);
59
60         const seleccionados: typeof pokemonesConIV = [];
61         const tiposUsados = new Set<string>();
62
63         for (const p of pokemonesConIV) {
64           if (!tiposUsados.has(p.tipo)) {
65             seleccionados.push(p);
66             tiposUsados.add(p.tipo);
67           }
68           if (seleccionados.length === 6) break;
69         }
70
71         return {
72           nombre: jugador.nombre,
73           pokemones: seleccionados
74         };
75       }));
76
77     } catch (e) {
78       console.error('Error al parsear:', e);
79     }
80
81     console.log(JSON.stringify(jugadores, null, 2));
82
83     res.render('pages/index', {
84       tokens: tokensToSend,

```

